Step-by-Step MySQL Deployment on Ubuntu EC2

Step 1: Launch EC2 Instance

- Go to AWS Console → EC2 → Launch Instance
- Choose **Ubuntu Server 20.04 LTS**
- Select instance type: `t2.micro` (Free Tier)
- Configure:
  - Add storage (default 8GB+)
  - Add security group rule:
    - **TCP port 22** (for SSH)
    - **TCP port 3306** (for MySQL access)
- Launch with key pair (for SSH access)

Step 2: SSH into the Instance

```
ssh -i "your-key.pem" ubuntu@<Public-IP>
```

Step 3: Update and Install MySQL

```
sudo apt update
sudo apt install mysql-server -y
```

Step 4: Secure MySQL Installation

```
sudo mysql_secure_installation
```

Follow prompts:

- Set root password
- Remove anonymous users
- Disallow root remote login
- Remove test database
- Reload privileges

Step 5: Enable MySQL Remote Access (Optional)

1. Edit MySQL config:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

2. Find line:

```
bind-address = 127.0.0.1
```

Change to:

```
bind-address = 0.0.0.0
```

3. Restart MySQL:

```
sudo systemctl restart mysql
```

Step 6: Create Remote-Accessible User

```
sudo mysql -u root -p
```

In MySQL shell:

```
CREATE USER 'remoteuser'@'%' IDENTIFIED BY 'yourpassword';
GRANT ALL PRIVILEGES ON *.* TO 'remoteuser'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

Step 7: Allow MySQL in UFW (If UFW is enabled)

```
sudo ufw allow 3306
sudo ufw allow ssh
sudo ufw enable
```

Step 8: Connect from External Client

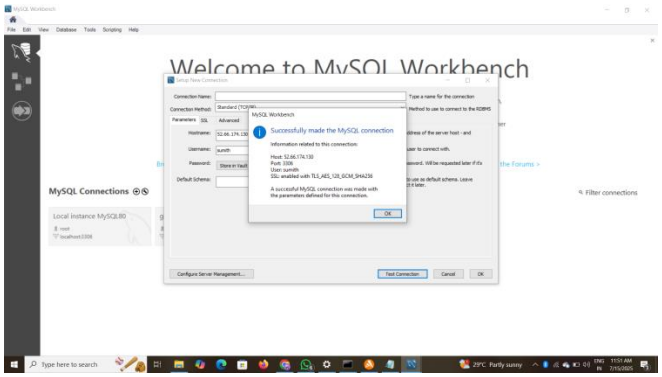
Use MySQL Workbench or:

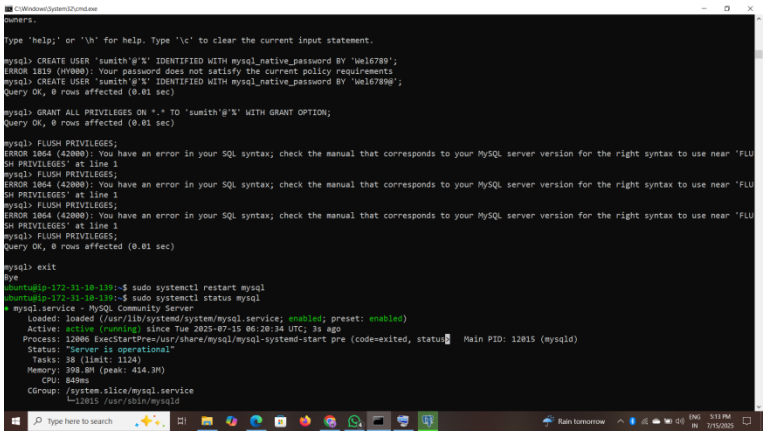
```
mysql -h <EC2-Public-IP> -u remoteuser -p
```

Make sure port 3306 is open in the EC2 security group and firewall (UFW).

Security Tips

- Avoid using `root` for remote access.
- Use strong passwords.
- Consider setting up an SSH tunnel for secure access instead of exposing port 3306.
- For production, consider using **Amazon RDS** instead of manually installing MySQL.


MySQL Workbench
File Edit View Database Tools Scripting Help
Welcome to MySQL Workbench
Setup New Connection
Connection Name:
Connection Method:
Parameters SSL Advanced
Hostname:
Username:
Password:
Store in Vault
Default Schema:
Type a name for the connection
Method to use to connect to the RDBMS
MySQL Workbench
Successfully made the MySQL connection
Information related to this connection:
Host: 52.66.174.130
Port: 3306
User: sumith
SSL: enabled with TLS_AES_128_GCM_SHA256
A successful MySQL connection was made with the parameters defined for this connection.
OK
MySQL Connections
Local instance MySQL80
root
localhost:3306
Filter connections
Configure Server Management...
Test Connection
Cancel
OK
Type here to search
29°C Partly sunny
ENG IN
11:57 AM 7/15/2025


C:\Windows\System32\cmd.exe
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE USER 'sumith'@'%' IDENTIFIED WITH mysql_native_password BY 'Wel6789';
ERROR 1819 (HY000): Your password does not satisfy the current policy requirements
mysql> CREATE USER 'sumith'@'%' IDENTIFIED WITH mysql_native_password BY 'Wel67890';
Query OK, 0 rows affected (0.01 sec)
mysql> GRANT ALL PRIVILEGES ON *.* TO 'sumith'@'%' WITH GRANT OPTION;
Query OK, 0 rows affected (0.01 sec)
mysql> FLUSH PRIVILEGES;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'FLU
SH PRIVILEGES' at line 1
mysql> FLUSH PRIVILEGES;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'FLU
SH PRIVILEGES' at line 1
mysql> FLUSH PRIVILEGES;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'FLU
SH PRIVILEGES' at line 1
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)
mysql> exit
Bye
ubuntu@ip-172-31-10-139:~$ sudo systemctl restart mysql
ubuntu@ip-172-31-10-139:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
Active: active (running) since Tue 2025-07-15 06:20:14 UTC; 3s ago
Process: 12006 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status
Main PID: 12015 (mysqld)
Status: "Server is operational"
Tasks: 38 (limit: 1124)
Memory: 398.8M (peak: 414.3M)
CPU: 849ms
CGroup: /system.slice/mysql.service
└─12015 /usr/sbin/mysqld
Type here to search
Rain tomorrow
ENG IN
5:13 PM 7/15/2025

PostgreSQL Deployment on Ubuntu EC2 – SME Guide

This guide is intended for professionals who want to **deploy, configure, and secure PostgreSQL** on an **AWS Ubuntu EC2 instance**.

Goals

- Deploy PostgreSQL on Ubuntu EC2.
- Enable secure remote access.
- Understand key configuration files and tuning.
- Follow best practices for production.

Environment Requirements

Component	Requirement
OS	Ubuntu 20.04+
Cloud	AWS EC2
Database	PostgreSQL 14+
Access	SSH, Port 5432 (TCP) open

Step 1: Launch EC2 & Configure

1. Launch EC2 (Ubuntu 20.04 or later)
2. Select `t2.micro` (or appropriate)
3. Security Group:
   - TCP 22 → SSH
   - TCP 5432 → PostgreSQL access (your IP only!)

Step 2: SSH into the Instance

```
bash
Copy code
ssh -i "your-key.pem" ubuntu@<EC2-Public-IP>
```

Step 3: Install PostgreSQL

```
bash
Copy code
sudo apt update
sudo apt install postgresql postgresql-contrib -y
```

Check PostgreSQL status:

```
bash
Copy code
sudo systemctl status postgresql
```

Step 4: Access PostgreSQL CLI

Switch to the default Postgres user:
bash
Copy code
```
sudo -i -u postgres
psql
```
Exit:
sql
Copy code
```
\q
```

Step 5: Create User and Database
bash
Copy code
```
sudo -i -u postgres
createuser --interactive
createdb mydb
```
Or via psql:
bash
Copy code
```
psql
CREATE USER myuser WITH ENCRYPTED PASSWORD 'mypassword';
CREATE DATABASE mydb OWNER myuser;
GRANT ALL PRIVILEGES ON DATABASE mydb TO myuser;
\q
```

Step 6: Enable Remote Access (Optional)
1. Edit `postgresql.conf`:
bash
Copy code
```
sudo nano /etc/postgresql/14/main/postgresql.conf
```
Change:
ini
Copy code
```
# listen_addresses = 'localhost'
```
To:
ini
Copy code
```
listen_addresses = '*'
```
2. Edit `pg_hba.conf`:
bash
Copy code
```
sudo nano /etc/postgresql/14/main/pg_hba.conf
```
Add at the end:
css
Copy code
```
host    all             all             0.0.0.0/0            md5
```
Or for specific IP:
css
Copy code
```
host    all             all             192.168.1.100/32     md5
```

3. Restart PostgreSQL

```
bash
Copy code
sudo systemctl restart postgresql
```

Step 7: Open Firewall Ports (UFW)

If UFW is used:

```
bash
Copy code
sudo ufw allow 5432/tcp
```

Step 8: Connect Remotely

Use tools like **pgAdmin**, **DBeaver**, or CLI:

```
bash
Copy code
psql -h <EC2-Public-IP> -U myuser -d mydb
```

Security Best Practices

Practice	Description
Use strong DB passwords	Avoid `12345`, `admin`, etc.
Restrict IPs in SG	Avoid 0.0.0.0/0 in production
Disable unused ports	Only open SSH & 5432
Regular backups	Use `pg_dump`, or automate
Use limited DB roles	No root-level rights
Audit & logs	Monitor `/var/log/postgresql`

Bonus: Backup and Restore Commands

Backup:

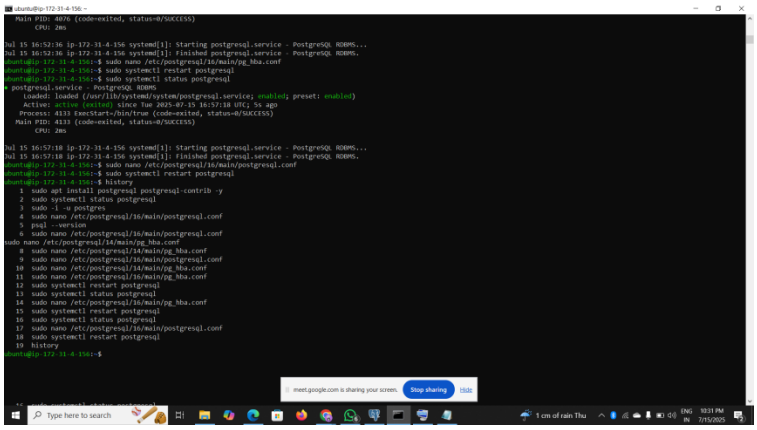
```
bash
Copy code
pg_dump -U myuser -h localhost mydb > backup.sql
```

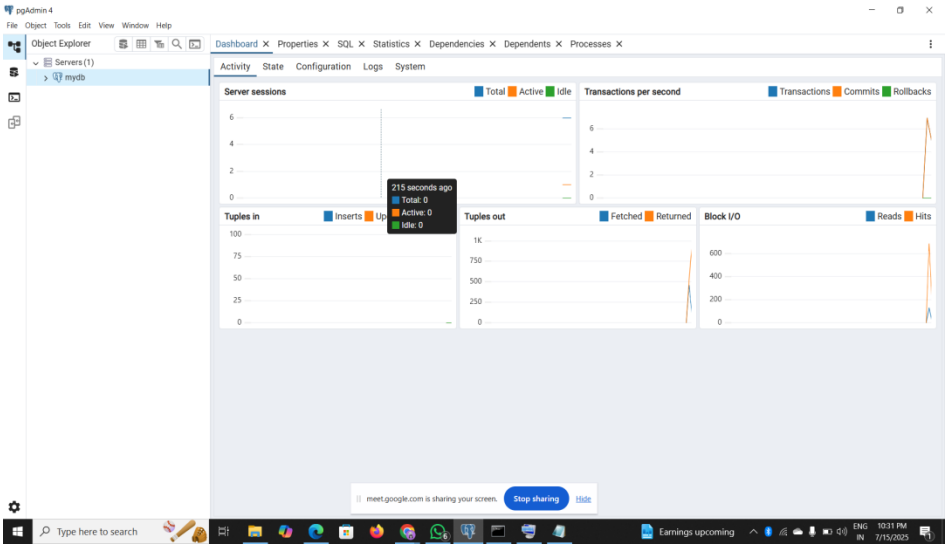
Restore:

```
bash
Copy code
psql -U myuser -h localhost mydb < backup.sql
```

PostgreSQL Config Files Location

File	Path
Main Config	/etc/postgresql/14/main/postgresql.conf
Authentication	/etc/postgresql/14/main/pg_hba.conf
Logs (by default)	/var/log/postgresql/





1. NoSQL Database Basics (Amazon DynamoDBvs Relational DBs)

Amazon DynamoDB Overview

- **Type:** Fully managed NoSQL database by AWS.
- **Data Model:** Key-Value and Document-based.
- **Schema:Schema-less** — each item in a table can have different attributes.
- **Performance:** Single-digit millisecond response times at any scale.
- **Scalability:Auto-scaling** for read/write throughput.
- **Storage:** SSD-backed, multi-AZ replication for durability.

DynamoDBvs Relational Databases (e.g., MySQL)

Feature	DynamoDB (NoSQL)	MySQL (Relational DB)
Schema	Flexible schema	Fixed schema (tables with defined columns)
Query Language	API-based (No SQL language)	SQL (Structured Query Language)
Transactions	Limited ACID support	Full ACID compliance
Scaling	Horizontal (auto-scalable)	Vertical (manual tuning/scaling)
Use Case	High-volume, low-latency applications	Structured data and relational integrity

---

2. Jenkins Ports & Admin Basics

Default Jenkins Port

- **Port:**8080
- Used for web-based access to the Jenkins dashboard.
- Can be customized by editing /etc/default/jenkins or jenkins.xml (Windows).

Trigger a Simple Build Job – Tutorial

Step-by-Step:

1. **Access Jenkins UI:**http://<your-server-ip>:8080
2. **Login as Admin**
3. **Create Job:**
   - Click on “New Item”
   - Enter job name → Select **Freestyle Project** → Click OK
4. **Configure Build:**
   - In **Build** section, click **Add build step**
   - Choose “Execute Shell” or “Windows batch command”
   - Enter a command (e.g., echo Hello Jenkins)
5. **Save & Build:**
   - Click **Save**
   - Click **Build Now**
6. **Check Console Output:**
   - Click on the build # → Console Output

---

3. Advanced MySQL Security

Password Policies – validate_password Plugin

- Enforce strong password rules in MySQL.

Enable Plugin:

INSTALL PLUGIN validate_password SONAME 'validate_password.so';

Check Policy:

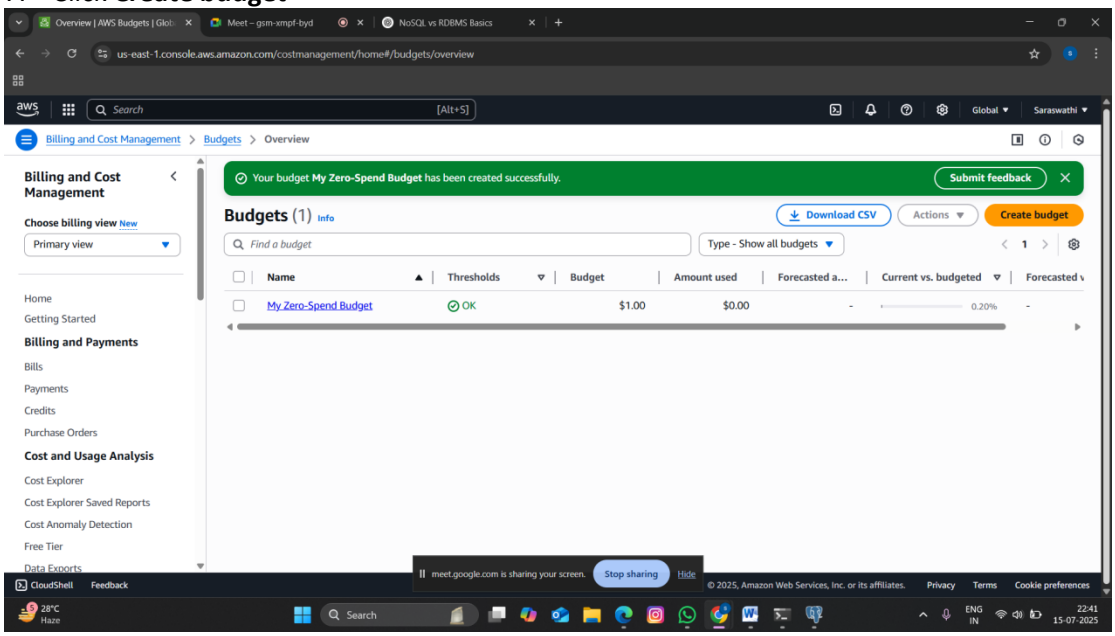
SHOW VARIABLES LIKE 'validate_password%';

Set Complexity:

SET GLOBAL validate_password.policy = MEDIUM;
SET GLOBAL validate_password.length = 12;
Effect: Prevents users from setting weak passwords.

Role-Based Access Control (RBAC)
Create Role:
CREATE ROLE 'readonly';
Grant Privileges:
GRANT SELECT ON mydb.* TO 'readonly';
Assign Role to User:
GRANT 'readonly' TO 'user1'@'localhost';
SET DEFAULT ROLE 'readonly' TO 'user1'@'localhost';
Benefits: Easier to manage permissions at scale.

---

4. AWS Billing Alerts Setup
Set Up AWS Budgets Billing Alert
Steps:

1. Go to **AWS Console → Billing → Budgets**
2. Click **Create budget**
3. Choose **Cost budget**
4. Set Budget Amount (e.g., $0.01 for free-tier users)
5. Set **Email alert** threshold (e.g., 100% of budget)
6. Add your email in **Notifications**
7. Click **Create budget**